```
        }

        return Task.CompletedTask;                                    ❺
    }
}
```

❶ The handler implements AuthorizationHandler<T>.

❷ You must override the abstract HandleRequirementAsync method.

❸ Checks whether the user has the IsBannedFromLounge claim

❹ If the user has the claim, fails the requirement by calling Fail. The whole policy fails.

❺ If the claim wasn't found, does nothing

In most cases, your handlers will either call `Succeed()` or will do nothing, but the `Fail()` method is useful when you need a kill switch to guarantee that a requirement won't be satisfied.

NOTE Whether a handler calls `Succeed()`, `Fail()`, or neither, the authorization system always executes all the handlers for a requirement and all the requirements for a policy, so you can be sure your handlers will always be called.

The final step to complete your authorization implementation for the app is to register the authorization handlers with the DI container, as shown in the following listing.

Listing 24.11 Registering the authorization handlers with the DI container

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddAuthorization(options =>

    options.AddPolicy(
        "CanEnterSecurity",
        policyBuilder => policyBuilder
            .RequireClaim(Claims.BoardingPassNumber));
    options.AddPolicy(
        "CanAccessLounge",
        policyBuilder => policyBuilder.AddRequirements(
```